

Attention Based & Blended RAG for NL2SQL

A PROJECT REPORT
SUBMITTED IN PARTIAL FULFILMENT OF THE
REQUIREMENTS FOR THE DEGREE OF

Master of Technology (Online)
IN
Data Science and Business Analytics

BY
Shunmuga Janani A



Faculty of Engineering
Indian Institute of Science
Bangalore – 560 012 (INDIA)

June, 2026

Declaration of Originality

I, **Shunmuga Janani A**, with SR No. **13-19-02-19-52-24-1-24615** hereby declare that the material presented in the project report titled

Attention Based & Blended RAG for NL2SQL

represents original work carried out by me at **Intel Corporation** as part of the project credit requirements of the **Master of Technology (Online)** degree in **Data Science and Business Analytics** at the Indian Institute of Science, between **August, 2024** to **June, 2026**.

With my signature, I certify that:

- I have not manipulated any of the data or results.
- I have not committed any plagiarism of intellectual property. I have clearly indicated and referenced the contributions of others.
- I have explicitly acknowledged all collaborative research and discussions.
- I have understood that any false claim will result in severe disciplinary action.
- I have understood that the work may be screened for any form of academic misconduct.



Date: 15th June 2026

Student Signature

In our capacities as internal project guide and faculty mentor of the above-mentioned work, we certify that the above statements are true to the best of our knowledge and we have carried out due diligence to ensure the originality of the report.



Internal Guide Name: *Sakthivel Subramanian*

Internal Guide Signature

Organisation: *Intel Corporation*



Faculty Mentor Name: *Shalabh Bhatnagar*

Faculty Mentor Signature

Department: *Data Science and Business Analytics*

© Shunmuga Janani A

June, 2026

All rights reserved

Acknowledgements

I would like to first thank Amit Baj and Sakthivel Subramanian from Intel for their mentorship, feedback and for sharing practical enterprise use cases that helped extend the NL2SQL solution beyond simple query generation. Their inputs were especially valuable in broadening the project toward testcase generation, lineage-aware reasoning and incident RCA workflows.

I would also like to express my sincere gratitude to Professor Deepak Subramani for teaching from the very basics of transformers to advanced agentic AI concepts through highly engaging and insightful classes. I am equally grateful to Professor Shalabh Bhatnagar for his guidance in reinforcement learning and for helping shape a rigorous way of thinking about intelligent systems. Their teaching greatly influenced the way I approached the design and evaluation of this work.

Finally, I would like to thank my husband and my mother for running the family with care and for constantly encouraging me to pursue and complete my degree. Their support, patience and belief in me made it possible to balance work, learning and family responsibilities throughout this journey.

Abstract

Enterprise NL2SQL systems often fail not because large language models cannot produce SQL syntax, but because they are unable to retrieve the correct schema evidence before generation begins. This challenge becomes more severe in enterprise environments where schema ambiguity is high, business concepts are duplicated across domains and foreign-key relationships are missing, incomplete or not explicitly maintained.

This project presents QueryOps as a hybrid retrieval-augmented generation platform for NL2SQL that also supports testcase generation, lineage-aware reasoning, cross-environment data comparison and incident root-cause analysis. The RAG implementation combines BM25 lexical retrieval, dense vector search, maximal marginal relevance diversification, attention-based reranking, relationship-aware context expansion, guarded SQL generation and repair loops for failed execution.

The report explains how these components work together to overcome two major barriers in enterprise NL2SQL: schema ambiguity and missing join relationships. Schema ambiguity is addressed through blended retrieval, reranking and transformation-logic summaries derived from DevOps-managed SQL assets, while missing foreign-key relationships are addressed through relationship-aware context construction that supplies executable join paths before SQL generation. The same grounded architecture also injects lineage context to support operational tasks beyond business query answering.

The evaluation strategy covers retrieval quality, SQL correctness, conversational usefulness, safety behavior and operational robustness. The testing across representative scenarios showed satisfactory behavior in schema grounding, follow-up handling and executable SQL generation. This report therefore positions the work not merely as a chatbot implementation, but as a

foundation for a measurable and extensible enterprise AI system.

Contents

Acknowledgements	i
Abstract	ii
Contents	iv
List of Figures	vii
List of Tables	viii
Acronyms, Abbreviations and Symbols	ix
1 Introduction	1
1.1 Background and Motivation	1
1.2 Problem Statement	2
1.3 Project Objectives	3
1.4 Contributions of the Work	3
1.5 Organization of the Report	4
2 Literature Review	5
2.1 Retrieval-Augmented Generation	5
2.2 Lexical and Dense Retrieval	5
2.3 Diversification and Reranking	6
2.4 Hybrid and Blended Retrieval	6

2.5	Attention-Guided Context Selection	6
2.6	Research Gap	7
3	Methodology and Implementation	8
3.1	System Overview	8
3.2	Hybrid Retrieval-Augmented SQL Generation	8
3.2.1	Embedding generation with AI summaries	8
3.2.2	Transformation logic and lineage context	10
3.2.3	Pipeline design	10
3.2.4	Why pure top-k retrieval is insufficient	11
3.2.5	Relationship-aware context construction	11
3.2.6	Chain-of-thought prompting for SQL generation	13
3.2.7	SQL fixer loop	14
3.3	Specialized Agents	14
3.3.1	Testcase Generator Agent	14
3.3.2	Data Comparison Agent	15
3.3.3	Incident RCA Agent	15
3.3.4	Conversation history logic	18
3.4	Implementation Characteristics	19
4	Results and Discussions	20
4.1	Immediate Outcomes	20
4.2	Testing Strategy	21
4.3	Quantitative Metrics	21
4.4	Observed Failure Categories	23
4.5	Impact on Testing and Incident Management	23
4.6	Advantages Over the Existing Databricks Genie Space Solution	24
4.7	Discussion	25
5	Conclusions	26

List of Figures

3.1	Multi-agent QueryOps architecture for NL2SQL and adjacent enterprise workflows.	9
3.2	SQL generation pipeline.	12
3.3	Data comparison flow across environments.	16
3.4	Incident RCA chain used by QueryOps.	17
4.1	Observed evaluation metrics for the current QueryOps prototype.	22

List of Tables

4.1	Comparison of QueryOps and Databricks Genie Spaces for enterprise schema reasoning and operational intelligence.	24
-----	--	----

Acronyms, Abbreviations and Symbols

This section summarizes the commonly used acronyms, abbreviations and symbols referenced in the report.

Acronyms

AST Abstract Syntax Tree

BM25 Best Matching 25 lexical relevance model

CIF Common implementation framework

LLM Large Language Model

MMR Maximal Marginal Relevance

NL2SQL Natural Language to SQL

PII Personally Identifiable Information

RAG Retrieval-Augmented Generation

RCA Root Cause Analysis

RLHF Reinforcement Learning from Human Feedback

SFT Supervised Fine-Tuning

Abbreviations

DEV Development environment

FK Foreign Key

PROD Production environment

QA Quality Assurance environment

SQL Structured Query Language

Symbols

K Number of retrieved candidate objects considered during ranking

$top-k$ Top-ranked retrieved items under a retrieval function

Chapter 1

Introduction

1.1 Background and Motivation

Enterprise analytics teams increasingly expect natural-language access to data, but practical SQL work in large organizations remains slow, fragmented and expensive. Analysts, testers and support engineers still spend considerable time searching across schemas, understanding object meaning, identifying valid joins, reading transformation logic from external code repositories and manually reconciling results across systems. In many environments, these activities are spread across multiple tools and are frequently tied to a single database product, which makes knowledge reuse difficult and prevents teams from having a one-stop assistant for data operations.

The motivation for QueryOps is therefore broader than text-to-SQL alone. A business question may require multiple fact and dimension tables, but a naive top- k retrieval strategy can fail simply because the required answer depends on more objects than the fixed retrieval window returns. If a question requires six tables and the system only surfaces the top three candidates, the generated SQL may still be syntactically correct while remaining semantically incomplete. This limitation becomes more severe in enterprise data estates where similar business concepts are distributed across many schemas and where foreign-key metadata is often unavailable or weakly maintained.

The same retrieval challenge appears in adjacent operational workflows. Test engineers often

1. INTRODUCTION

need to manually understand an object’s business logic, design testcase scenarios, write validation SQL, execute it in each environment, compare outputs and then document the verdict. Support engineers performing incident RCA must log in to multiple systems, inspect tickets, search lineage, open DevOps assets and reason through upstream transformations before even forming a probable root cause. These manual workflows increase turnaround time, increase dependence on a small number of subject-matter experts and raise the operating cost of enterprise data programs.

In this setting, a grounded retrieval-augmented platform offers value in two ways. First, it reduces query-authoring effort for NL2SQL by retrieving joinable schema context, transformation summaries and lineage evidence before SQL generation. Second, it extends the same retrieval foundation to testcase generation, cross-environment data comparison and incident RCA, thereby converting isolated data-engineering tasks into a unified AI-assisted workflow.

1.2 Problem Statement

This project addresses the problem of building an enterprise AI assistant that can generate executable SQL and support surrounding data-operations workflows with the same contextual backbone. The core problem is not merely language generation. Rather, it is the difficulty of assembling complete and trustworthy context from large schemas, missing relationship metadata, DevOps-managed SQL logic, historical incidents and lineage dependencies.

Three recurring obstacles dominate this problem. The first is schema ambiguity: semantically similar tables exist across domains, while lexically different names may refer to the same business concept. The second is relationship incompleteness: many enterprise databases do not maintain usable foreign-key constraints, which makes valid join-path construction difficult. The third is operational fragmentation: existing assistants are frequently scoped to a single database or semantic layer and do not bring together SQL generation, testcase automation, data comparison, lineage reasoning and incident RCA in one distributed solution.

The consequence is substantial manual overhead. For testing, engineers must author comparison SQL by hand and repeat validation across DEV, QA and PROD. For incident analysis, engineers must switch between ticketing systems, lineage tools, database consoles and code

1. INTRODUCTION

repositories before reaching a resolution. For NL2SQL, even strong language models can fail when the retrieved context is incomplete, especially when a pure top- k approach excludes one or more required tables. QueryOps is proposed to solve this by using query decomposition, hybrid retrieval, attention-based reranking, relationship-aware context building, lineage expansion, AST-based logic summarization and repair loops that improve the final answer when first-pass execution fails.

1.3 Project Objectives

The primary objectives of the project are as follows:

1. design a hybrid retrieval-augmented architecture for schema-grounded enterprise NL2SQL;
2. extend the same retrieval foundation to testcase generation, cross-environment comparison and incident root-cause analysis;
3. incorporate relationship-aware and lineage-aware context so that retrieved objects are not only relevant, but also complete context aware RAG which can assist in any kind of RCA.

1.4 Contributions of the Work

The contributions of this project can be summarized in four parts. First, it proposes a blended retrieval pipeline that combines query decomposition, BM25 lexical search, dense vector retrieval, MMR diversification and attention-style reranking for schema-grounded NL2SQL. Second, it introduces relationship-aware context expansion so that relevant objects are also joinable and complete enough for multi-table enterprise questions. Third, it extends the same retrieval foundation to testcase generation, multi-environment comparison and incident RCA by incorporating lineage and AST-based logic summaries from DevOps-managed assets. Finally, it frames the system as an enterprise productivity platform with measurable time savings, cost savings and operational reuse rather than as a standalone chatbot.

1. INTRODUCTION

1.5 Organization of the Report

The remainder of this report is organized as follows. Chapter 2 reviews the literature on retrieval-augmented generation, lexical and dense retrieval, reranking and hybrid retrieval. Chapter 3 presents the QueryOps architecture, the SQL generation flow, the SQL fixer loop and the supporting agents for testing, comparison, lineage reasoning and incident analysis. Chapter 4 discusses the testing strategy, quantitative results, qualitative advantages and enterprise impact of the system. Chapter 5 concludes the report and outlines the next steps for human-feedback collection, dataset construction and improved reranking or alignment.

Chapter 2

Literature Review

2.1 Retrieval-Augmented Generation

Retrieval-Augmented Generation (RAG) combines parametric language modeling with non-parametric evidence retrieval so that model outputs can be grounded in external knowledge sources [1]. In enterprise database applications, this paradigm is especially useful because schema metadata, lineage records and code artifacts change more frequently than the underlying language model. RAG therefore provides a practical way to keep system behavior aligned with the current data environment.

2.2 Lexical and Dense Retrieval

Dense retrieval methods, such as Dense Passage Retrieval, improve semantic matching by learning embedding spaces in which related questions and passages are close to one another [2]. However, dense retrieval alone can be insufficient in enterprise environments where object names, abbreviations and business terminology have domain-specific exact-match importance. BM25 remains a strong lexical baseline because it captures exact token overlap and term-frequency effects in a transparent way [3]. In schema retrieval, lexical and dense signals are often complementary rather than interchangeable.

2. LITERATURE REVIEW

2.3 Diversification and Reranking

Even when relevant candidates are retrieved, redundancy can reduce the usefulness of the final context. Maximal Marginal Relevance (MMR) addresses this problem by selecting candidates that balance relevance and diversity [4]. After diversified retrieval, a reranking stage can further improve precision by jointly evaluating the user query and each candidate object. Attention-based reranking methods, such as BERT passage reranking, have shown that final-stage contextual scoring can significantly improve retrieval quality over coarse first-stage ranking alone [5].

2.4 Hybrid and Blended Retrieval

Recent work has shown that sparse and dense retrieval signals often work best when deliberately combined. Hybrid retrieval approaches improve robustness by preserving lexical precision while also capturing semantic similarity [6, 7]. More recent blended RAG work also argues for combining semantic retrieval with query-oriented retrieval strategies to improve grounding quality in downstream generation systems [8]. This line of work strongly motivates the retrieval strategy used in QueryOps, where lexical, dense, diversified and reranked signals are all combined before SQL generation.

2.5 Attention-Guided Context Selection

Large context windows do not automatically lead to better grounded generation. Attention-guided RAG methods argue that context quality improves when systems actively prune or re-score retrieved evidence before generation [9]. For enterprise NL2SQL, this principle is highly relevant because schema stores often contain many near-duplicate objects, historical versions, or partially relevant artifacts. A reranking and pruning stage therefore helps keep the final prompt both compact and actionable.

2. LITERATURE REVIEW

2.6 Research Gap

Most published retrieval work focuses on open-domain question answering or general document retrieval. Enterprise NL2SQL introduces additional constraints: retrieved objects must not only be relevant, but also joinable; transformation logic may be more informative than metadata labels alone; and the same retrieval layer must often support adjacent tasks such as testcase generation and incident RCA. While hybrid retrieval and reranking are studied broadly in information retrieval, an attention-based blended RAG design specifically targeted at enterprise NL2SQL is not widely established as an end-to-end approach.

Existing enterprise assistants also do not typically inject AST-derived transformation logic, code-level lineage evidence and past-incident knowledge into the same reasoning loop. As a result, incident RCA is often performed without a complete understanding of upstream objects and their corresponding code logic, even though that information is critical for trustworthy diagnosis. This gap motivates a system-level design that retrieves not only tables and columns, but also relationship structure, lineage, transformation summaries, historical incidents and conversation memory so the platform can support both SQL generation and operational reasoning.

Chapter 3

Methodology and Implementation

3.1 System Overview

QueryOps is designed as a multi-agent enterprise platform for natural-language SQL generation and operational intelligence. The platform supports NL2SQL query generation, testcase generation, cross-environment comparison, lineage-aware reasoning, conversation-aware follow-up handling and incident root-cause analysis using a shared retrieval backbone. The central design principle is that downstream reasoning quality depends on whether the system can assemble the right schema, logic and lineage evidence before generation begins.

3.2 Hybrid Retrieval-Augmented SQL Generation

3.2.1 Embedding generation with AI summaries

The retrieval layer depends on a curated embedding store rather than raw object names alone. For each ECA object retrieved from the information schema, together with the active SAP HANA objects, QueryOps first collects structural metadata such as table name, column names, data types and schema location. It then performs lightweight data profiling over the columns, including statistics such as minimum value, maximum value, null count, distinct count and representative top- k value distributions where useful. These signals are passed to the LLM so that it can generate a business-oriented description of the object and a concise intent summary

3. METHODOLOGY AND IMPLEMENTATION

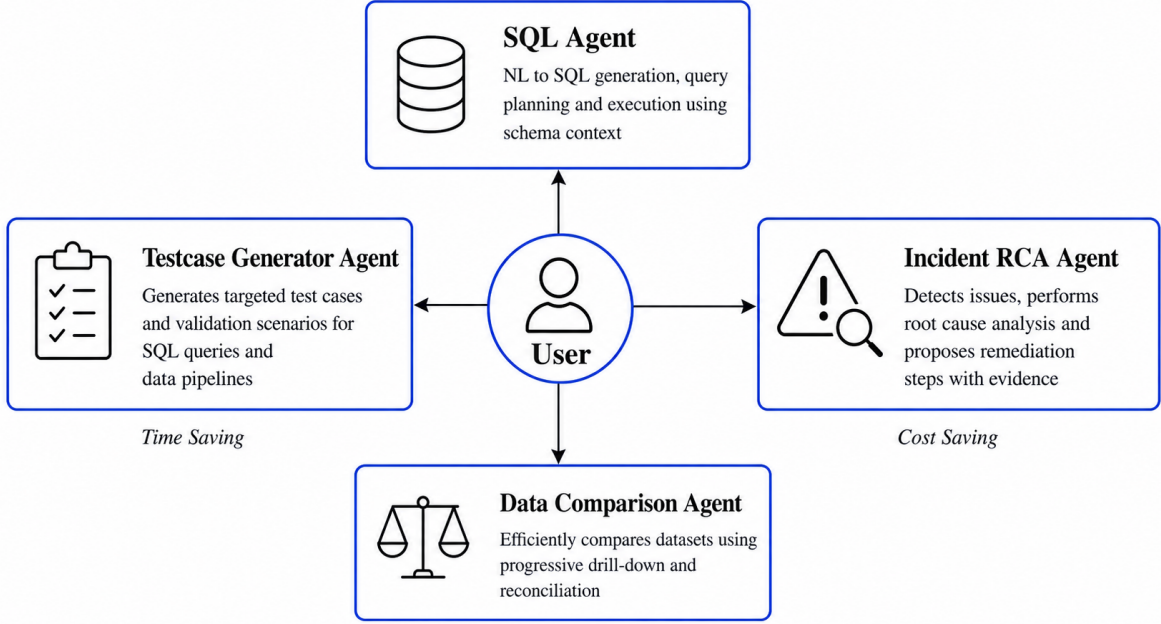


Figure 3.1: Multi-agent QueryOps architecture for NL2SQL and adjacent enterprise workflows.

for each important column.

This AI-generated summarization step is important because enterprise table names are often abbreviated, overloaded or too technical to be useful for direct semantic retrieval. By combining the raw table and column names with the LLM-generated table description, column summaries and object intent, the system creates a richer text representation before embedding. The resulting embeddings are stored in PostgreSQL alongside the original metadata so that later retrieval can operate over both structural and semantic evidence. In practice, this means the vector search is not matching only on names, but on a compressed business-facing explanation of what the object contains and how it is likely to be used.

The same preparation stage also supports relationship construction. During metadata processing, likely key relationships are derived by comparing column-level signals across objects and supplementing them with successful historical query patterns. As a result, the embedding layer and the relationship layer are built together: one improves semantic recall and the other improves downstream joinability.

3. METHODOLOGY AND IMPLEMENTATION

3.2.2 Transformation logic and lineage context

In many enterprise systems, object names alone are insufficient to explain business meaning. For the ECA environment, the SQL logic that builds objects is maintained in DevOps rather than inside the warehouse itself. QueryOps therefore builds transformation-logic metadata before retrieval time and sends that metadata forward as part of the reasoning context rather than sending raw SQL alone.

The transformation-logic preparation pipeline follows a structured sequence. First, the object-building SQL is collected from the source assets. Next, the SQL is parsed using `sqlglot`. From this parsed representation, the system derives five metadata products: the abstract syntax tree (AST), the join graph, column lineage, output grain, and a compact logic summary. These metadata elements are then stored in the vector database together with the object representation so that retrieval can match not only on schema names, but also on transformation behavior and lineage evidence.

At generation time, the retrieved transformation metadata is injected into context along with the object summaries and relationship evidence. This means the SQL agent receives a compact representation of how an object is built, how joins are formed, which upstream columns feed the output, what grain the object represents, and what the high-level business logic is. This is more useful than passing raw SQL alone because it exposes the reasoning-relevant structure in a normalized and prompt-efficient form.

For HANA-backed objects, direct dependency information can be derived from system metadata. After dependency extraction, lineage is stored and traversed through recursive database queries so that the reasoning layer can bring upstream and downstream context into SQL generation, testcase generation, and incident RCA. This makes the retrieved context not only relevant, but also causally informative.

3.2.3 Pipeline design

The SQL generation flow begins with a natural-language question. Instead of sending the question directly to a single vector search stage, the system first builds an embedding and then decomposes the request into retrieval intents. This decomposition identifies likely fact-

3. METHODOLOGY AND IMPLEMENTATION

table needs, dimension-table needs, measure cues, filter conditions and business entities. The decomposition step is important because enterprise questions often contain both grain-defining and lookup-style requirements, and these should not be retrieved with the same priority profile.

For each intent, the system executes BM25 lexical retrieval, dense vector retrieval and MMR diversification in parallel. The candidate results are unioned and deduplicated before passing through an attention-based reranker. The reranker scores each candidate by considering the user question together with schema name, object description and relevant column evidence, which makes the final seed-object set more reliable than either lexical or vector similarity alone.

3.2.4 Why pure top-k retrieval is insufficient

A pure top- k retrieval pipeline is not sufficient for enterprise NL2SQL because relevance alone does not guarantee completeness. A question that requires six tables cannot be answered correctly if the system only keeps the top three candidate objects. Likewise, a vector search can return semantically similar but operationally wrong objects, especially when two tables belong to nearby embedding neighborhoods but represent different business entities. This is why QueryOps treats retrieval as a staged decision process instead of a single nearest-neighbor lookup.

The blended design improves this in three ways. BM25 protects lexical precision when names matter, vector search improves semantic recall when names differ from business phrasing and MMR reduces redundancy so that the candidate set covers more of the actual question. Attention-based reranking then filters the combined list into seed objects that best match the intended business task.

3.2.5 Relationship-aware context construction

After the seed objects are selected, QueryOps expands them into a joinable schema context. This context includes key relationships mined during the metadata preparation stage, along with successful historical join evidence from prior queries. The goal is not only to retrieve relevant tables, but to provide a complete join path that the SQL generator can actually execute. This step is particularly important in enterprise platforms where database constraints are absent or

3. METHODOLOGY AND IMPLEMENTATION

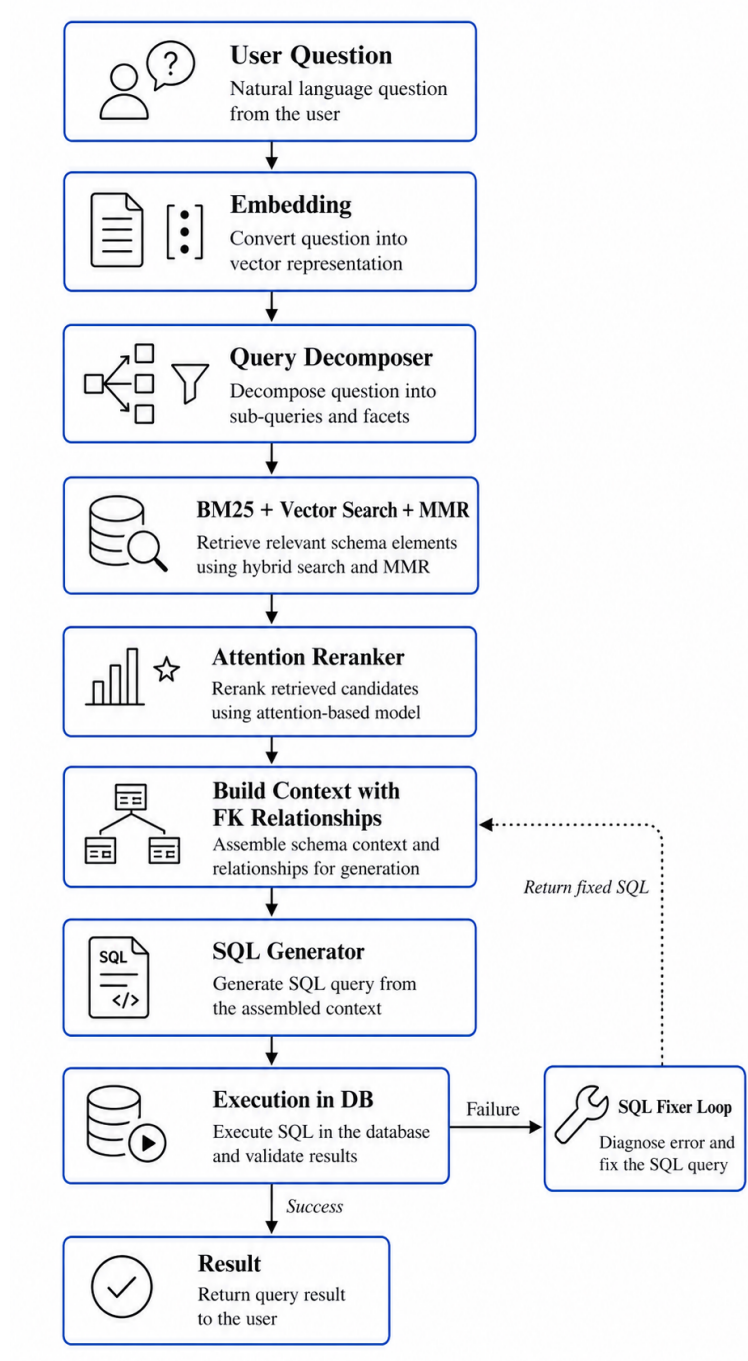


Figure 3.2: SQL generation pipeline.

3. METHODOLOGY AND IMPLEMENTATION

not trustworthy even though business relationships still exist in practice.

By incorporating relationship-aware context, the system avoids a common enterprise failure mode in which the right fact table is retrieved but the supporting dimensions, bridge tables, or valid join keys are missing. This relationship expansion is therefore directly tied to both join accuracy and final result accuracy.

3.2.6 Chain-of-thought prompting for SQL generation

The final SQL prompt is structured as a guided reasoning flow rather than a single unconstrained generation request. The model is implicitly led through a sequence of reasoning steps: identify the candidate tables, select the required columns, determine the join path, decide the aggregation logic and then apply the business filters. This ordering is useful because enterprise SQL errors often arise from skipping directly to query text before table grain and relationship choices have been stabilized.

In practice, the SQL agent receives the user question together with the schema context, join options, object summaries and dialect instructions for the target system. The prompt then encourages stepwise reasoning over five decisions: (1) identify the primary fact and supporting dimension tables, (2) choose the output and filter columns grounded in schema evidence, (3) define the joins using the supplied relationship context, (4) apply the aggregation logic that matches the requested grain and (5) apply the date, business or environment filters required by the question. Only after these decisions are resolved is the final SQL produced.

This chain-of-thought style prompting improves controllability in two ways. First, it keeps the model anchored to the retrieved context instead of inventing columns or unsupported relationships. Second, it provides a better interface to the SQL fixer loop, because execution failures can be traced back to a particular decision stage such as wrong table choice, wrong join path, or missing filter. The result is a more diagnosable and repairable SQL-generation workflow than a one-shot prompting scheme.

3. METHODOLOGY AND IMPLEMENTATION

3.2.7 SQL fixer loop

The SQL generation stage is guarded by a dialect-aware SQL fixer loop. When execution fails, the error message is inspected and a repair attempt is generated automatically. The repair process can rewrite syntax, adjust column or object references, or re-execute RAG with the execution error as an additional hint so that a different context set is selected. The process runs iteratively for up to three attempts before escalating to a wider retrieval path.

Authorization failures are treated differently from semantic SQL failures. If the query fails due to access restrictions, the loop does not continue retrying because the issue is not solvable through query rewriting. For all other execution failures, the loop improves robustness by converting first-pass failure into guided re-generation rather than terminating the user workflow.

3.3 Specialized Agents

3.3.1 Testcase Generator Agent

Test engineers traditionally perform data validation through a slow manual workflow. They first study the object and its business purpose, then identify testcase scenarios, write SQL for each testcase, run the SQL in one or more environments, compare the results and finally document the verdict. In practice, this can take about one week for a single object when the logic is complex and when multiple environments need to be validated.

The QueryOps testcase generator reduces this effort through a two-phase generation pipeline. In the first phase, the system generates a structured testcase plan using schema metadata, column types, nullability, AI-generated object summaries, lineage context and transformation-logic summaries. In the second phase, each testcase is converted into executable SQL using the same SQL generation backbone as the NL2SQL agent. Because lineage and code logic are already present in context, the produced testcases are more targeted toward verifying whether the expected data flow from upstream objects to downstream views or tables is preserved.

The resulting workflow can reduce object-level validation effort from roughly seven days to one day, which corresponds to an approximate time saving of **60%**. This is valuable not only because it accelerates testing, but also because it standardizes testcase depth and documenta-

3. METHODOLOGY AND IMPLEMENTATION

tion quality across testers. The generated cases can cover completeness, uniqueness, validity, referential checks, business-rule validation and transformation-aware reconciliation.

3.3.2 Data Comparison Agent

Cross-environment comparison is another expensive manual task, especially when teams need to compare DEV, QA and PROD outputs for large tables. QueryOps first aligns comparable objects and identifies the shared columns that should participate in comparison. If the source and target are within the same environment or engine, the agent can generate SQL based on set-difference logic such as `EXCEPT`. If the comparison spans different systems or very large tables, the agent uses a Merkle-tree-style hashing flow to avoid full data movement.

The Merkle-tree logic is designed for scale. It begins with a root-level row-count and bucket aggregation check on both sides. Rows are grouped into a fixed set of hash buckets and each bucket is summarized using deterministic hashes. Buckets whose hashes match are eliminated immediately, while only mismatched buckets are recursively split into smaller bucket groups. The drill-down continues until the remaining mismatched subset is small enough for leaf-level fetch and Python-side reconciliation.

This strategy is effective because it avoids pulling all records across systems. Instead of transferring entire tables, the agent compares a small number of bucketed hash summaries and only reads detailed rows where differences actually exist. When mismatches are sparse, the amount of data moved is dramatically smaller than a full table comparison, which makes the agent practical for large enterprise datasets.

3.3.3 Incident RCA Agent

Incident resolution in enterprise data operations is usually slow because evidence is fragmented. Engineers often begin with a ticket in ServiceNow, then search historical incidents, inspect the affected objects, trace lineage, open DevOps assets and manually infer where the transformation broke. QueryOps converts this into a six-step retrieval-and-reasoning chain: parse the incident, recall similar past incidents, scope likely objects, traverse lineage, fetch logic and synthesize a root-cause explanation with remediation guidance.

3. METHODOLOGY AND IMPLEMENTATION

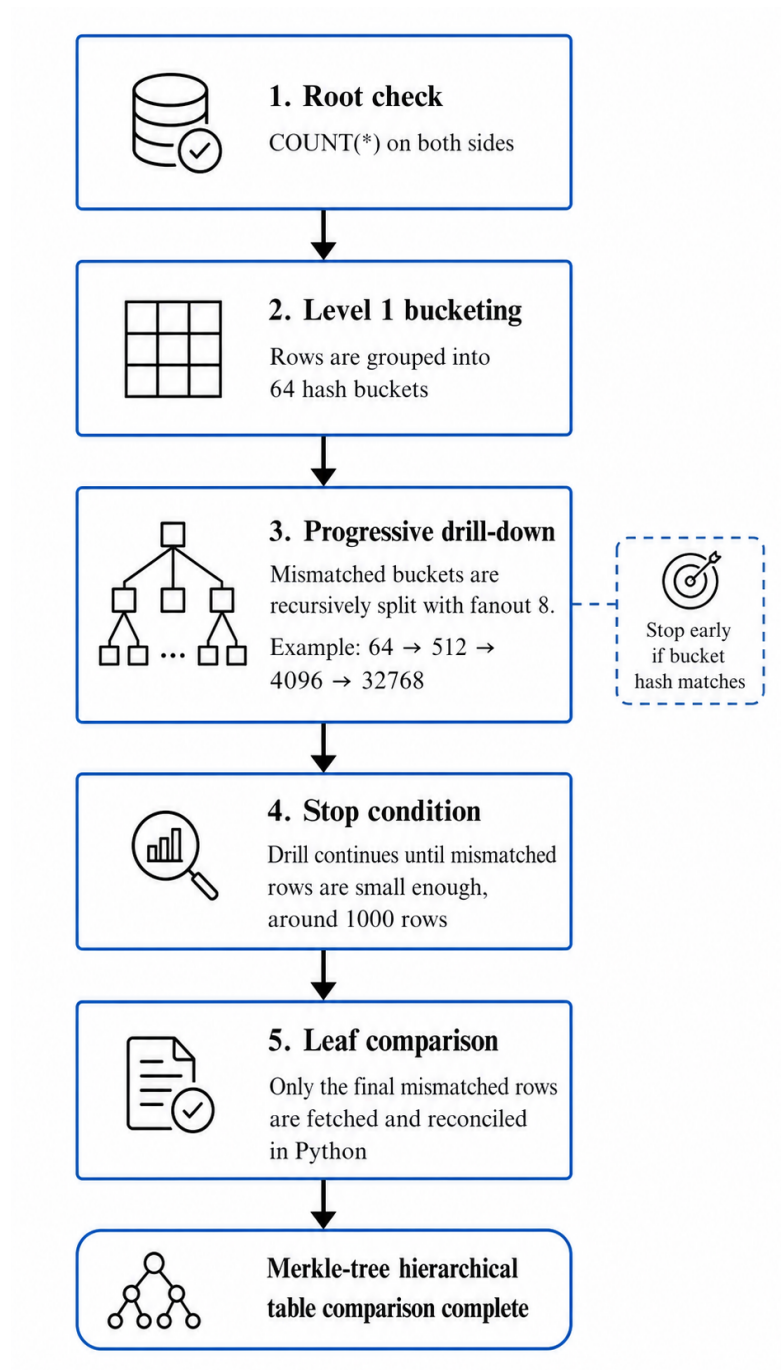


Figure 3.3: Data comparison flow across environments.

3. METHODOLOGY AND IMPLEMENTATION

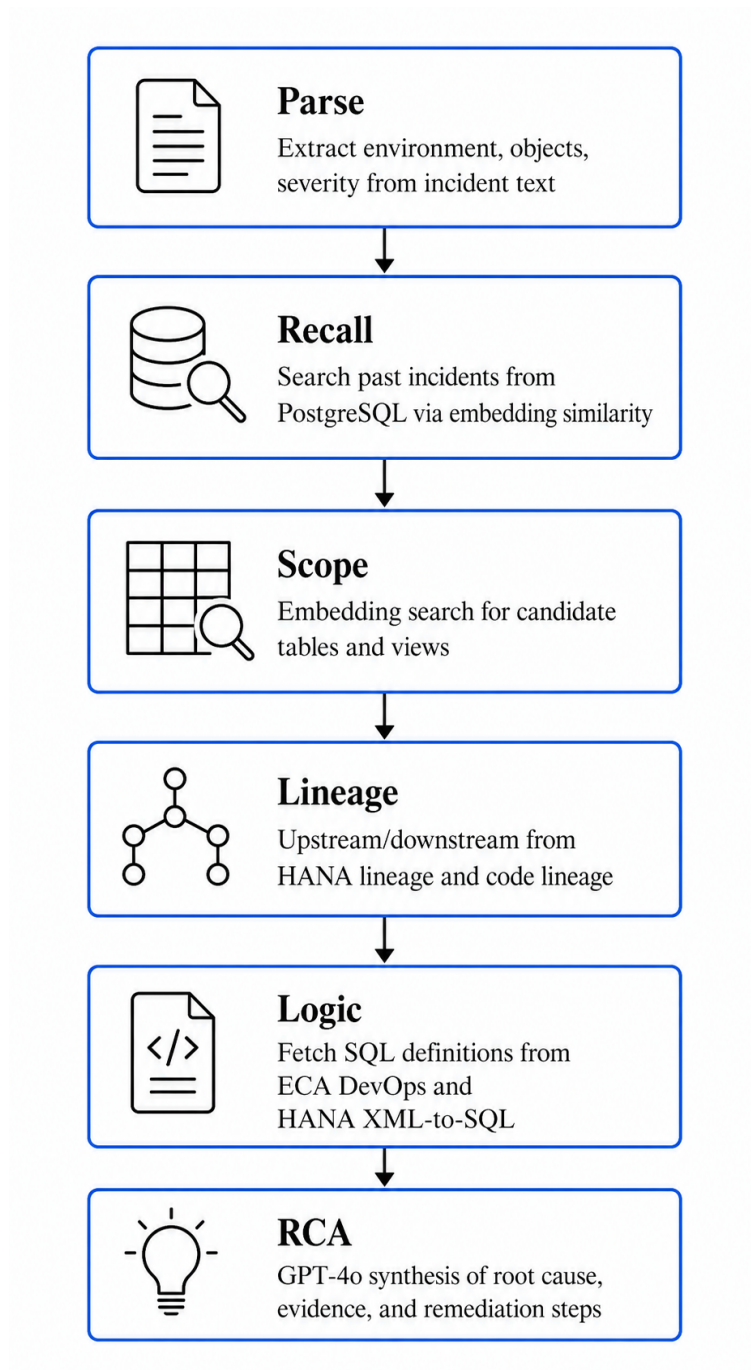


Figure 3.4: Incident RCA chain used by QueryOps.

3. METHODOLOGY AND IMPLEMENTATION

The platform also uses incident history as a knowledge source. Resolved incidents, activity logs and close notes are summarized into compact AI-generated solution narratives, embedded and stored for future retrieval. This allows the agent to retrieve not just similar ticket text, but also the corrective pattern that was previously applied. Once historical incident evidence is combined with lineage and AST-based transformation logic, the assistant can generate a more grounded resolution path than a ticket-only search process.

Even if AI does not fully replace human support, reducing repetitive first-pass triage, evidence gathering and context switching can materially reduce the amount of specialist time spent per incident. The practical gain is not only lower cost exposure, but also faster recovery and more consistent support quality across time zones and teams.

3.3.4 Conversation history logic

Enterprise SQL questions are often conversational rather than one-shot. Users ask a base question, refine the grain, change a filter or refer back to the prior answer using short follow-up phrases. QueryOps therefore stores structured conversation memory instead of relying only on plain-text dialogue history. The memory contains the previous query state, the previous SQL, the matched tables used in the last answer and the recent user-assistant turn history.

At the UI layer, recent turns are retained in session state and sent with a stable conversation identifier. At the backend layer, PostgreSQL stores both a latest snapshot and a longer history table, along with the structured SQL and matched-table evidence. A vector index supports semantic lookup over older conversation states. During a follow-up turn, the SQL agent receives both the immediate session context and semantically retrieved historical context, allowing it to reuse prior schema grounding instead of rediscovering everything from scratch.

This design is preferable to a narrative-only summary because structured memory preserves the exact SQL, the last evidence set and the intended grain of the query. As a result, follow-up requests such as “now show only last month” or “use the same objects but group by plant” can be resolved more reliably.

3. METHODOLOGY AND IMPLEMENTATION

3.4 Implementation Characteristics

At an implementation level, the system integrates enterprise metadata, vector retrieval, code-aware logic extraction, lineage traversal, incident memory and LLM-driven reasoning into a single application flow. The architecture is intentionally reusable: the same retrieval backbone supports SQL generation, testcase planning, data comparison and incident RCA with different orchestration logic layered on top. This is what makes QueryOps a platform design rather than a task-specific prompt chain.

Chapter 4

Results and Discussions

4.1 Immediate Outcomes

The immediate outcome of the proposed architecture is a more reliable enterprise path for schema-grounded NL2SQL and adjacent data-operations use cases. By combining lexical precision, semantic recall, reranking, relationship-aware context building, lineage expansion and guarded execution, the system improves the chance that generated SQL is both executable and business-relevant. The same retrieval backbone also enables a broader class of workflows that otherwise require repeated manual navigation across databases, ticketing systems, lineage tools and DevOps repositories.

From an operational perspective, the strongest benefits appear in time savings and context consolidation. For testcase generation, a workflow that traditionally takes about one week per object can be reduced to about one day when the assistant generates the testcase plan, SQL validations and comparison logic using lineage and code context. This corresponds to an approximate **60%** reduction in turnaround time. For incident RCA, the system reduces the amount of manual effort required to log in to different systems, gather upstream dependencies, open object logic and connect historical incidents to current failures.

The architecture also improves enterprise reuse. Instead of deploying one assistant for SQL, another for support and separate manual practices for testing and comparison, the organization can build on one retrieval platform that serves multiple downstream tasks. This increases the

4. RESULTS AND DISCUSSIONS

return on metadata preparation, lineage extraction and incident-memory construction because the same context can be reused across different agents.

4.2 Testing Strategy

The testing strategy was designed to evaluate more than surface SQL text. Exact string comparison is not sufficient in enterprise NL2SQL because two SQL queries may differ syntactically while still being semantically equivalent and because an answer can look valid even when the upstream schema evidence is wrong. The evaluation was therefore structured as a staged pipeline that tests whether the system retrieved the correct objects, selected the correct join path, produced executable SQL and returned the intended result.

The primary testcase source was the previously documented validation material maintained by business functional consultants for ECA development. These assets already contained business-oriented testcase descriptions and validation SQL. They were normalized into a common JSON format covering question text, expected tables, joins, filters, grouping behavior and golden SQL. To broaden scenario diversity, a smaller set of additional cases was generated externally and curated into the same schema before automated evaluation.

The evaluation also included operational scenarios beyond NL2SQL. For testcase generation, the generated SQL assertions were checked against known validation expectations. For incident management, previously resolved incidents that had not yet been embedded were used as an evaluation pool and the AI-generated RCA was compared against historical close notes and resolution patterns. For conversational behavior, follow-up prompts were tested against prior SQL state and retrieved conversation memory to verify whether the system preserved context correctly.

4.3 Quantitative Metrics

The evaluation metrics were chosen to reflect business correctness rather than only model fluency. Retrieval quality was measured using Table Recall@ K , Column Recall@ K and Precision@ K . Join quality was measured separately because enterprise errors frequently arise from choosing the wrong relationship path even when the correct seed object is present. Execution accuracy

4. RESULTS AND DISCUSSIONS

measured whether the final SQL ran successfully on the target platform, while result accuracy measured whether the answer matched the expected business output.

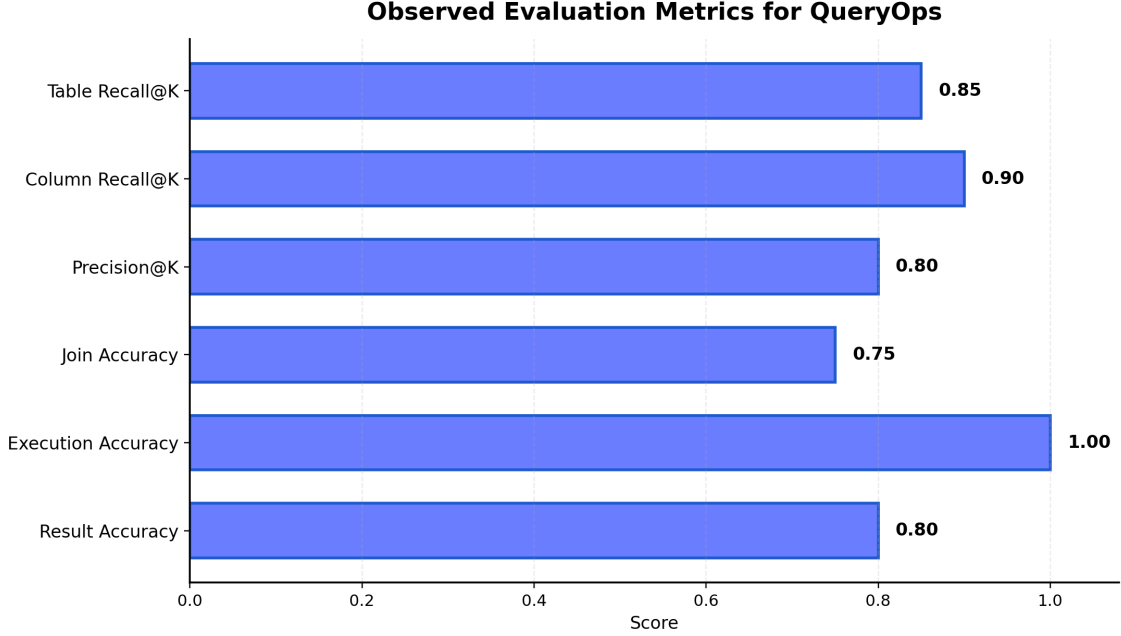


Figure 4.1: Observed evaluation metrics for the current QueryOps prototype.

The observed metrics were as follows:

1. Table Recall@ K = 0.85;
2. Column Recall@ K = 0.90;
3. Precision@ K = 0.80;
4. Join Accuracy = 0.75;
5. Execution Accuracy = 1.00;
6. Result Accuracy = 0.80.

Note: The reported values are rounded to the nearest multiple of 5.

These results show that the hybrid retrieval pipeline is able to recover most of the schema evidence needed to answer user questions. The strong execution accuracy indicates that the

4. RESULTS AND DISCUSSIONS

guarded generation and fixer logic produce structurally valid SQL. However, the lower join accuracy confirms that relationship selection remains one of the hardest enterprise failure modes, especially when several semantically plausible join paths exist. This gap between execution accuracy and result accuracy is precisely why relationship-aware context and lineage-aware retrieval are critical.

4.4 Observed Failure Categories

The main failure categories observed during evaluation were wrong-table selection, wrong join-path selection, hallucinated columns, wrong aggregation, missing business filters, wrong grouping grain and execution errors. Among these, wrong joins were the most important because they can preserve syntactic validity while changing the meaning of the answer. This again supports the core thesis of the project: enterprise NL2SQL quality depends more on context completeness and join-path reasoning than on text generation quality alone.

4.5 Impact on Testing and Incident Management

The testing impact is significant because the assistant changes both speed and quality of validation. A tester no longer has to start from a blank page by manually understanding the object, designing validation scenarios, authoring SQL and comparing outputs across environments. Instead, the system can generate a test plan grounded in column metadata, business summaries, lineage and code logic, then turn those tests into executable SQL. Because the agent has access to upstream and downstream context, it is better positioned to propose testcases that actually verify whether the intended transformation flow is preserved.

The incident-management impact is similarly substantial. In the existing workflow, engineers repeatedly spend time signing in to different tools, reading similar tickets, locating affected objects and tracing lineage manually. QueryOps reduces this repeated effort by retrieving past incidents, summarizing prior fixes, pulling lineage and injecting AST-parsed logic into one reasoning context. Even partial automation of first-pass triage and evidence assembly can therefore reduce support effort materially while also shortening mean investigation time.

4. RESULTS AND DISCUSSIONS

4.6 Advantages Over the Existing Databricks Genie Space Solution

Many business-facing NLQ systems depend on a small curated semantic space and metadata annotations. QueryOps is designed for a different enterprise setting in which transformation logic, lineage and DevOps-managed SQL assets are required for trustworthy generation and support reasoning. Table 4.1 summarizes the advantages of the proposed design.

Capability Area	QueryOps Model	Databricks Genie Spaces
Object Limit	Our approach is not designed around a fixed small-space semantic curation model; it can retrieve from a broader enterprise object estate, then narrow context dynamically using hybrid retrieval.	Databricks Genie Spaces are constrained to 30 tables or views per space as of May 21, 2026, which can limit coverage for enterprise domains where answering a single business question may require broader schema context and lineage expansion.
Schema grounding	Uses hybrid retrieval across metadata, code context, relationships and operational artifacts.	Officially described as using configured datasets, sample queries, instructions and annotated names and descriptions.
Transformation logic awareness	Can inject object-building SQL and AST-parsed transformation summaries from DevOps assets.	Documentation describes semantic configuration, but does not explicitly state that Genie Spaces use transformation logic behind each object.
Lineage-aware reasoning	Can traverse lineage and attach upstream and downstream dependencies into prompt context.	Databricks has lineage as a platform feature, but when transformation logic is maintained in DevOps and executed through generic CIF templates, both transformation-level reasoning and that DevOps-grounded lineage context are not directly available inside Genie Spaces.
Join-path construction	Uses pre-built FK inference and enterprise relationship context for SQL generation.	Documentation emphasizes annotated tables, columns and sample queries, not explicit relationship-expansion logic.
Enterprise operational reuse	The same architecture supports NL2SQL, testcase generation, RCA, lineage reasoning and data comparison.	Genie Spaces are primarily positioned for business self-serve analytics.

Table 4.1: Comparison of QueryOps and Databricks Genie Spaces for enterprise schema reasoning and operational intelligence.

4. RESULTS AND DISCUSSIONS

4.7 Discussion

The current results are encouraging because they show that retrieval-centered engineering can improve both SQL generation and neighboring operational tasks. At the same time, the metrics and error categories reveal that join-path selection remains the main bottleneck, which validates the design choice to invest in relationship-aware expansion, lineage context and error-driven fixer loops. The system therefore demonstrates not only a working NL2SQL assistant, but also a broader enterprise RAG pattern in which complete context matters more than isolated language fluency.

Chapter 5

Conclusions

This project presented QueryOps as a hybrid retrieval-augmented platform for enterprise NL2SQL and operational intelligence. The work argued that enterprise SQL quality depends fundamentally on retrieval quality, joinability, lineage context and transformation-logic awareness rather than on language generation alone. To address this, the platform combines BM25 lexical retrieval, dense vector search, MMR diversification, attention-style reranking, relationship-aware context construction and guarded SQL generation.

The report also showed that the same retrieval foundation can support more than business querying. Testcase generation, cross-environment comparison and incident RCA all benefit from grounded schema retrieval, lineage traversal and code-aware context. This broader applicability makes the platform more valuable as an enterprise engineering system than as a standalone chatbot.

The next steps are to collect structured human feedback, construct a gold benchmark dataset and quantify performance through controlled experiments. This future work should compare lexical-only, dense-only and hybrid retrieval, evaluate reranking and SQL-fixer effectiveness and assess whether stronger reranker training, SFT or RLHF-based preference alignment improves system behavior under realistic enterprise workloads.

References

- [1] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive nlp tasks. In *Advances in Neural Information Processing Systems*, 2020. URL <https://arxiv.org/abs/2005.11401>. 5
- [2] Vladimir Karpukhin, Barlas Oguz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. Dense passage retrieval for open-domain question answering. In *Proceedings of EMNLP*, 2020. URL <https://arxiv.org/abs/2004.04906>. 5
- [3] Stephen Robertson and Hugo Zaragoza. The probabilistic relevance framework: Bm25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389, 2009. doi: 10.1561/15000000019. 5
- [4] Jaime Carbonell and Jade Goldstein. The use of mmr, diversity-based reranking for re-ordering documents and producing summaries. In *Proceedings of the 21st Annual International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 335–336, 1998. URL https://ir.webis.de/anthology/1998.sigirconf_conference-98.43/. 6
- [5] Rodrigo Nogueira and Kyunghyun Cho. Passage re-ranking with bert. *arXiv preprint arXiv:1901.04085*, 2019. URL <https://arxiv.org/abs/1901.04085>. 6
- [6] Yi Luan, Jacob Eisenstein, Kristina Toutanova, and Michael Collins. Sparse, dense, and attentional representations for text retrieval. *Transactions of the Association for Computa-*

REFERENCES

- tional Linguistics*, 9:329–345, 2021. URL <https://aclanthology.org/2021.tacl-1.20/>. 6
- [7] Dohyeon Lee, Seung-won Hwang, Kyungjae Lee, Seungtaek Choi, and Sunghyun Park. On complementarity objectives for hybrid retrieval. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics*, pages 13357–13368, 2023. URL <https://aclanthology.org/2023.acl-long.746/>. 6
- [8] Kunal Sawarkar, Anish Mangal, and S. R. Solanki. Blended rag: Improving rag accuracy with semantic search and hybrid query-based retrievers. *arXiv preprint arXiv:2404.07220*, 2024. URL <https://arxiv.org/pdf/2404.07220>. 6
- [9] Yuhan Fang, Tianyang Sun, Yuxin Shi, and Xiang Gu. Attentionrag: Attention-guided context pruning in retrieval-augmented generation. *arXiv preprint arXiv:2503.10720*, 2025. URL <https://arxiv.org/abs/2503.10720>. 6